Module 5 In-Class Activity

University of Niagara Falls Canada
Fall 2024 Sql Databases (CPSC-500)
Instructor: Khaled Shahata

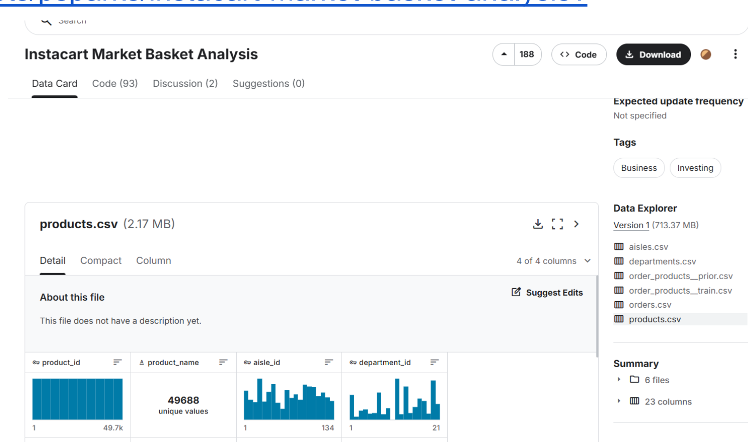
Overview

In this activity, we will work with real-world data from Instacart, an online grocery delivery service. The Instacart dataset provides valuable insights into customer shopping behaviors, including product categories, orders, and reordering trends. This dataset is available on Kaggle and includes multiple CSV files.

The aim of this activity is to **create the required tables**, **load data from the CSV files**, and **practice SQL syntax** related to Data Definition Language (DDL), Data Manipulation Language (DML), and Data Control Language (DCL) that you learned in module 5.

Dataset Download

This dataset is part of a machine learning contest. You can download the full dataset file from the following link:
https://www.kaggle.com/datasets/psparks/instacart-market-basket-analysis?select=products.csv



However, I prepared a **smaller version of the dataset** that you can download from the module content. The file is named **InstacartDataSample**.

Please note that **Kaggle** is a great resource for data professionals, providing access to a wide variety of real-world datasets, competitions, and community-driven learning to sharpen your data skills.

For this activity, we'll use the following files from the Instacart dataset:

- **aisles.csv:** Contains unique aisle IDs and names.
- **departments.csv:** Lists departments with IDs and names.
- **products.csv:** Contains product details, each associated with an aisle and a department.
- **orders.csv:** Includes data on each order, such as order time and user ID.
- **order_products.csv:** Links orders to products, showing which products were added to each order.

Let's Add Data to MySQL

Step 1:

First we need to **create** a database. Let's call it instacart_db. Don't forget to **use** it!
Answer:
CREATE DATABASE instacart_db;
USE instacart_db;

Step 2:

Let's introduce ourselves to different CSV files and then we will create the tables for them.

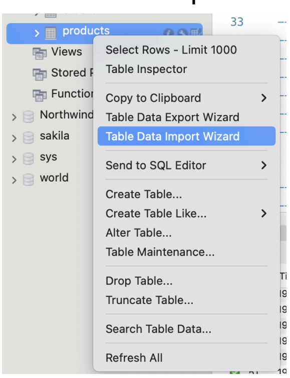
- **Products.csv:** This file stores details of individual products, each associated with a specific department and aisle. Each product has a name and a unique ID.
  - The product's name length cannot exceed 100 characters.
- **Departments.csv:** This file holds information about store departments, such as "produce" or "snacks." Each department has a unique identifier and a name.
  - The name of the department is limited to 50 characters.
- **Orders.csv:** Contains details about each customer order, such as the day of the week and the time it was placed. Each order has a unique identifier. The columns are:
  - order_id: A unique identifier for each order.
  - user_id: Identifies the customer who placed the order.
  - eval_set: Specifies the order dataset (e.g., "prior", "train," "test").
  - order_number: Indicates the sequence of the customer's orders (e.g., first, second).
  - order_dow: The day of the week (0-6) when the order was placed.
  - order_hour_of_day: The hour (0-23) when the order was placed.
  - days_since_prior_order: Days since the customer's previous order.
- **Aisles.csv:** Contains information about store aisles, grouping similar products together within the store layout. Each aisle has a unique ID.
  - The name of the isle is limited to 50 characters.
- **order_products.csv:** Acts as a junction table linking each order to the products it contains. This table also tracks the order in which products were added to the cart and whether the product was reordered.

Step 2.1: Given the relationships between these files, what do you recommend as the order of table creation to properly respect these dependencies?

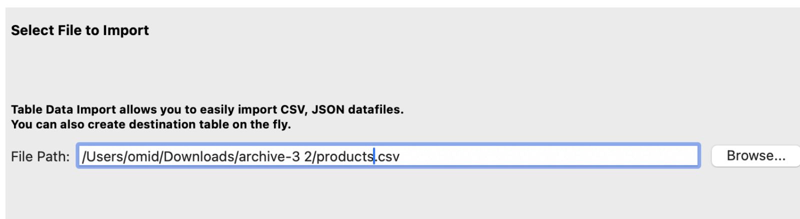
Step 2.2: Based on the ordering you suggested, define and create each of the tables.
Note 1: If an aisle or a department is deleted, any corresponding aisle_id or department_id values in the products table should be set to NULL.
Note 2: If a product or an order is deleted, all rows in the order_products table that reference the deleted product or order should also be removed.

Step 2.3: Import the data to the tables.

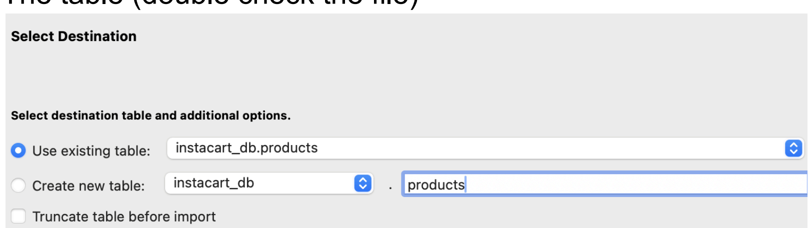
Select Data Import



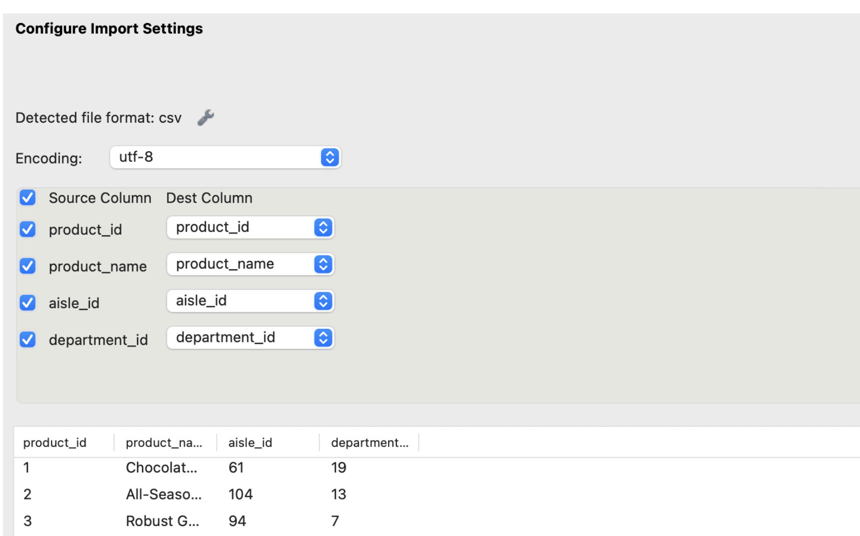
Select the file

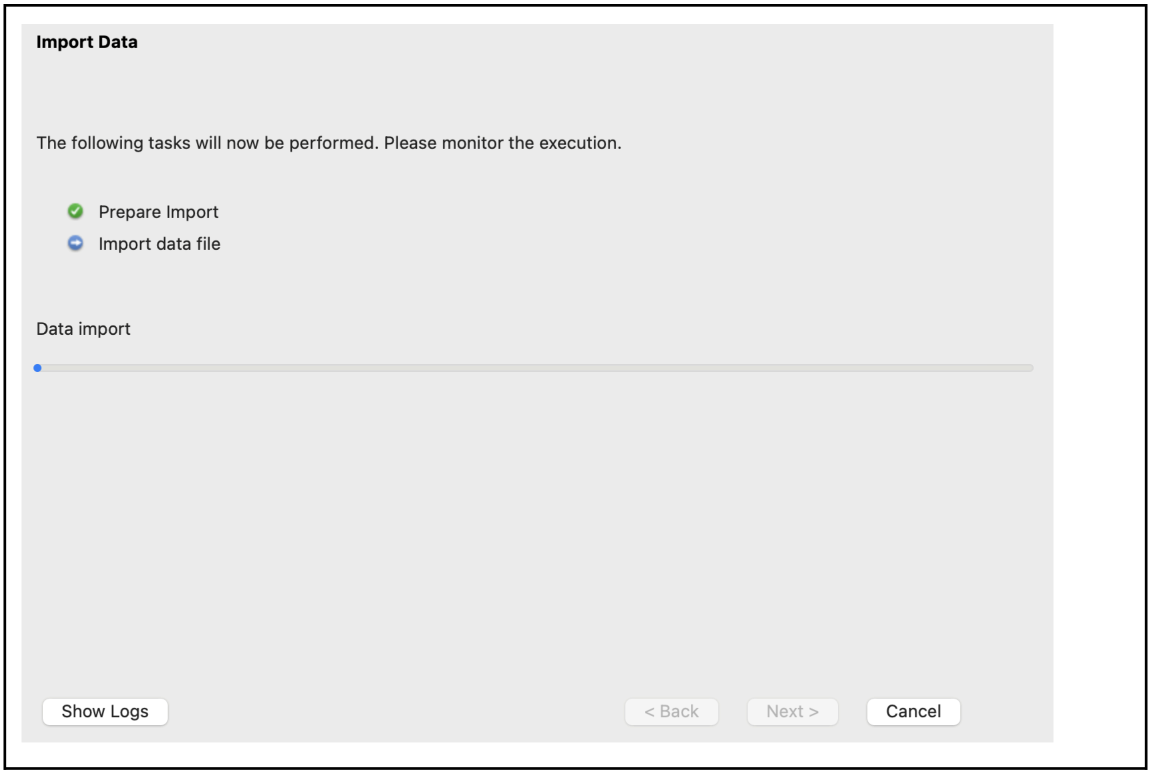


The table (double check the file)



Match the columns of file to columns of table





Step 2.4: Verify data import. Write a simple SQL query to check if the data is loaded properly.

SQL DML Statements

Insert, Update, and Delete Operations in SQL

After bulk importing the data, let's try some manual inserts with smaller entries into the tables.

Q1. Insert a new department called seasonal with ID = 22.

Q2. Insert a new product called Halloween Costume with ID = 50002 into the seasonal department and aisle 6.

Q3. Insert a new product called Tylenol with ID = 50001 into the department with id 23 and aisle 7. Did it go well? How can you solve the issue?

Q4. Delete a specific product with id=50002 directly from the products table.

Q5. Delete the 'Pharmacy' department that you added above, with id = 23. What happens to products in that department?

Q6. Delete order with order id 537159 from orders table. What happens to the order_products table?

Q7. Update the product with id = 50001 and change its department to 3.

SQL DDL Statements

Q8. Change the table to add a Discounted Price (decimal values with 5 total number of digits and 2 as the number of decimal places) Column to products.

Q9. Populate the new column by setting a discount as the discounted_price for all products. If the aisle id is 2 then set a 15% discount. Otherwise set a 10% discount.

Some Extra DDL

Q. Create a table called teams with a team_id and team_name. The primary key is team_id. We do not add data to this table.

Q. Create another table called team_members that has the member_id, team_id referring to the teams table, and member_name.

Q. Now that the table team_members refers to teams, try deleting the teams. Did it go well?How to solve the issue (I mean how to drop teams)?

SQL Views

Q10. Create a View for Frequently Ordered Products. Create a view named popular_products that shows each product's ID and the total number of times it was ordered, sorted by the most ordered products in descending order.

Q11. Create a View for Order Details. Create a view named order_details that displays each order's ID, user ID, product name, and the order in which products were added to the cart.

Q12. Query the views you created above.

Thank you,
Khaled